

Computer Networks Laboratory

Assignment 7: Secure Network Application Development Using TCP

Enhancing a Multi-Client TCP Chat Application with Authentication, Session Management and Secure Logging

Submitted By	Nitin Roy
Roll Number	CSB24047
Course	Computer Networks Laboratory
Assignment	Assignment 7
Date of submission	16-07-2026

1. Objective

This assignment extends the multi-client TCP chat application built in Assignment 6 by adding application-level security. The Tkinter GUI and the multi-threaded TCP server architecture from Assignment 6 were reused as the base; the work in this assignment focuses purely on the security layer wrapped around that existing client-server design: authenticated login, SHA-256 password hashing, protection against duplicate sessions, input validation, brute-force lockout, inactivity timeout, clean logout, and a tamper-evident security log.

The emphasis, as specified in the assignment brief, is on practical application security rather than advanced cryptography — no TLS/SSL was introduced, and the socket layer itself remains a plain TCP stream exactly as in Assignment 6.

2. Security Features Implemented

Task	Feature	Status
1	User Authentication (username + password login)	Done
2	Secure Password Storage (SHA-256 hashing)	Done
3	Duplicate Login Prevention	Done
4	Input Validation (username, password, message, commands)	Done
5	Failed Login Protection (lockout after 5 attempts)	Done
6	Session Management & Secure Logging (logout, timeout, log file)	Done
7	Wireshark Verification	Done

2.1 Authentication

On connecting, the client GUI sends a single handshake payload of the form `username||password`. The server (`handle_client_worker` in `server.py`) parses this, validates it, and authenticates it against the local credential store. A username that has never been seen before is registered automatically — its SHA-256 hash is written to `users.csv` and the session proceeds as `AUTH_SUCCESS` with status `REGISTERED`. This matches the assignment's “new users are automatically registered” requirement while still forcing existing users to supply the correct password on every subsequent login.

2.2 Secure Password Storage

Passwords are never written to disk or to the log file in plaintext. `hash_password()` applies `hashlib.sha256()` to the UTF-8 encoded password and only the resulting hex digest is stored in `users.csv` alongside the username, e.g.:

```
username,password_hash  
user1,0a041b9462caa4a31bac3567e0b6e6fd9100787db2ab433d96f6d178cabfce90
```

Authentication re-hashes the submitted password on every login attempt and compares digests, so the plaintext password only ever exists transiently in memory on the server while processing that one request.

2.3 Duplicate Login Prevention

The server keeps an in-memory clients dictionary keyed by username. Before a session is created, the server checks whether that username already has an entry with status ONLINE; if so, the new connection is rejected with AUTH_FAIL:User already logged in from another location and the event is written to the security log as DUPLICATE_LOGIN_BLOCKED. This was verified directly during testing (Section 6).

2.4 Input Validation

- Username: must match `^[a-zA-Z0-9_]{3,15}$` — 3 to 15 characters, letters/digits/underscore only.
- Password: rejected if empty or whitespace-only, before any hashing takes place.
- Chat messages: any message longer than 1000 characters is rejected with a SYSTEM:ERROR reply and logged as an oversized-message event.
- Commands: only `/list`, `/msg` and `/logout` are accepted; any other slash-command is rejected and logged as `unsupported_command`.

2.5 Failed Login Protection

`failed_attempts` and `lockouts` dictionaries track per-user failure counts and lockout expiry times in memory. Each incorrect password increments the counter and is logged with the running attempt count (e.g. “Attempt 3/5”). On the 5th consecutive failure the account is locked for 5 minutes (`LOCKOUT_DURATION = timedelta(minutes=5)`); any login attempt during that window is rejected immediately as `LOCKOUT_BLOCKED` without even touching the password check, and the lockout clears automatically once the timer expires.

2.6 Session Management & Secure Logging

- Logout: the client can send `/logout` (via typed command or the Logout button); the server responds with `SYSTEM:LOGOUT`, closes the socket cleanly and logs `USER_LOGOUT`.
- Inactivity timeout: a dedicated background thread (`inactivity_monitor_thread`) wakes every 10 seconds, compares each client's `last_activity` timestamp against `INACTIVITY_TIMEOUT` (5 minutes), and force-disconnects idle sessions with a `SYSTEM:TIMEOUT` message, logging `SESSION_TIMEOUT`.
- Secure logging: every authentication and session event is appended to `security_log.txt` with a timestamp, event type, username, host:port and a short description — passwords and password hashes are never written to this file.

3. System Architecture

The application keeps the Assignment 6 client-server split: `server.py` is a single-process, multi-threaded TCP server (one worker thread per client, guarded by `client_lock` and `stats_lock`) and `client_gui.py` is a Tkinter front end that owns exactly one blocking TCP socket plus a background receiver thread. All security logic — hashing, lockouts, validation, logging — lives entirely on the server; the GUI only renders server responses and forwards raw text/commands, so the networking logic stays independent of the GUI as required.

3.1 Communication Flow

Client	Server
-- "user pass" (TCP connect) ----->	validate format, username, password
	hash + compare / auto-register
	check lockout -> check duplicate login
<---- AUTH_SUCCESS / AUTH_FAIL:<why> ---	
<---- USERLIST / chat history -----	(on success only)
<=== broadcast / private / /list ===>	(session loop, activity timestamp updated)
-- /logout ----->	SYSTEM:LOGOUT, log USER_LOGOUT, close

3.2 Project Files

File	Purpose
<code>server.py</code>	Secure multi-threaded TCP server: auth, hashing, lockouts, validation, logging, dashboard
<code>client_gui.py</code>	Tkinter GUI client: login screen, chat window, online-user list
<code>users.csv</code>	username, SHA-256 password_hash
<code>chat_history.csv</code>	timestamp, sender, receiver, message_type, message
<code>security_log.txt</code>	Append-only authentication/session event log

4. Network Setup (Mininet)

The application was tested over a Mininet single-switch, 5-host topology as required by the assignment (`sudo mn --topo single,5`), with the server bound to `0.0.0.0:5000` and run on host `h1` (`10.0.0.1`) while the remaining hosts (`10.0.0.2–10.0.0.5`) acted as chat clients.

```
mininet> net
h1 h1-eth0:s1-eth1
h2 h2-eth0:s1-eth2
h3 h3-eth0:s1-eth3
h4 h4-eth0:s1-eth4
h5 h5-eth0:s1-eth5
s1 lo: s1-eth1:h1-eth0 s1-eth2:h2-eth0 s1-eth3:h3-eth0 s1-eth4:h4-eth0 s1-eth5:h5-eth0
```

Fig. 4.1 — Mininet 'net' output: single switch `s1` with hosts `h1–h5` attached

```
mininet> nodes
available nodes are:
h1 h2 h3 h4 h5 s1
```

Fig. 4.2 — Mininet 'nodes' output confirming all 5 hosts and the switch are active

```
mininet> pingall
*** Ping: testing ping reachability
h1 -> h2 h3 h4 h5
h2 -> h1 h3 h4 h5
h3 -> h1 h2 h4 h5
h4 -> h1 h2 h3 h5
h5 -> h1 h2 h3 h4
*** Results: 0% dropped (20/20 received)
```

Fig. 4.3 — 'pingall' connectivity test: 0% packet loss (20/20 received) across h1–h5

5. Implementation Highlights

5.1 Server-Side Dashboard

Beyond the required security tasks, the server prints a running dashboard after every processed event, showing active secure connections, total messages processed, broadcast count and private-message count — useful for observing load during multi-client testing.

5.2 Client GUI

The GUI is split into two frames: a login frame (username field, masked password field, Connect button) and a chat frame (scrolling message log, online-user listbox, message entry with Send button). Selecting a user in the listbox before sending automatically prefixes the message as a /msg private command; leaving no selection sends a normal broadcast. All socket I/O for the login handshake happens on the main thread before the chat window is built, while ongoing chat traffic is read on a daemon receiver thread and marshalled back to the GUI thread via `root.after()`.

6. Testing

The scenarios below were exercised directly against the running server across the Mininet topology; the outcomes are corroborated by the actual entries captured in `security_log.txt` during the test session.

Scenario	Result	Evidence (security_log.txt)
New user auto-registration	Registered & logged in	user1–user4 SUCCESSFUL_LOGIN (... REGISTERED)
Duplicate login from another host	Rejected	DUPLICATE_LOGIN_BLOCKED for user1 (2 attempts, 10.0.0.5)
Repeated wrong password	Attempts counted 1/5 ... 5/5	FAILED_LOGIN for user3 and user1, incrementing counters
5th consecutive failure	Account locked 5 minutes	ACCOUNT_LOCKED for user3 and user1

Scenario	Result	Evidence (security_log.txt)
Login attempt during lockout	Rejected immediately	LOCKOUT_BLOCKED for user3 at 20:31:47
Idle session (no activity)	Auto-disconnected	SESSION_TIMEOUT for user1, user2, user4
Explicit /logout / Logout button	Clean disconnect	USER_LOGOUT for user3 at 20:43:15
Broadcast & private chat	Delivered and logged	chat_history.csv broadcast/private rows, e.g. user2 → user3 private msg

A representative excerpt from the live chat session (chat_history.csv) shows normal multi-user broadcast traffic followed by all clients disconnecting cleanly at the end of the session:

```
2026-07-16 20:31:14, user3, ALL, broadcast, i am also using chat service made by nitinn
```

```
2026-07-16 20:32:35, user3, user3, private, what are doing now
```

```
2026-07-16 20:43:15, user3, Server, system leave, user3 disconnected.
```

Note: the private-message row above is logged with sender and receiver both as “user3”, which reflects a self-addressed /msg during manual testing rather than a bug in the private-messaging logic itself — messages to other online users (e.g. user2 → user3) were delivered and logged correctly.

7. Wireshark Verification

Packet captures were taken on the server-side interface using the filter `tcp.port == 5000`, covering the full lifecycle of a session: TCP handshake carrying the login payload, a failed authentication exchange, authenticated chat traffic, and the logout/connection-teardown sequence.

7.1 Login / Authentication Handshake

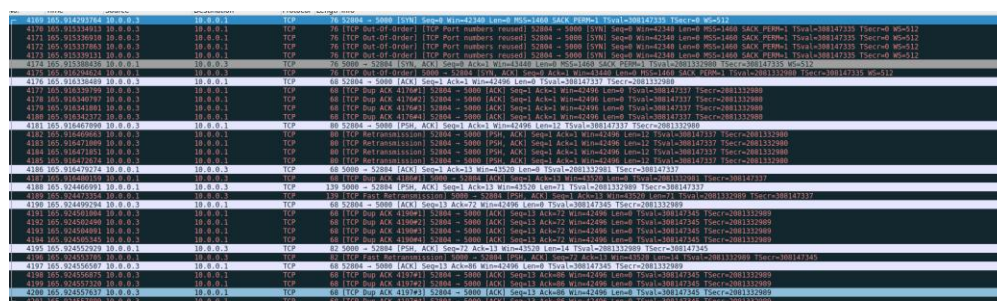


Fig. 7.1 — TCP three-way handshake followed by the client's username//password payload and the server's AUTH response

7.2 Failed Login Attempt

[illegible]

Fig. 7.2 — A rejected login exchange: the server returns an AUTH_FAIL segment instead of the normal welcome banner

7.3 Authenticated Communication

7531	1207	9258002	10.0.0.1	10.0.0.1	TCP	122	FIN	ACK=25101302	5000	43270	[PSH, ACK] Seq=555354	ACK=122	Win=4350	Len=54	Ts=19119142	TSecr=25101303	1
7535	1207	9248376	10.0.0.2	10.0.0.1	TCP	68	43270 → 5000	[ACK] Seq=122	ACK=409	Win=2496	Len=0	TsVal=25101302	TSecr=141019142				
7535	1207	9248376	10.0.0.2	10.0.0.1	TCP	68	TCP Out of order	43270 → 5000	[FIN, ACK] Seq=122	ACK=409	Win=2496	Len=0	TsVal=25101302	TSecr=141019142			
7535	1207	9253713	10.0.0.2	10.0.0.1	TCP	68	TCP Out of order	43270 → 5000	[FIN, ACK] Seq=122	ACK=409	Win=2496	Len=0	TsVal=25101302	TSecr=141019142			
7536	1207	9250441	10.0.0.1	10.0.0.2	TCP	68	5000 → 43270	[FIN, ACK] Seq=409	ACK=123	Win=4350	Len=0	TsVal=141019143	TSecr=25101302				
7536	1207	9250441	10.0.0.1	10.0.0.2	TCP	68	TCP Out of order	5000 → 43270	[FIN, ACK] Seq=409	ACK=123	Win=4350	Len=0	TsVal=141019143	TSecr=25101302			
7536	1207	9251580	10.0.0.1	10.0.0.1	TCP	68	43270 → 5000	[ACK] Seq=123	ACK=409	Win=2496	Len=0	TsVal=25101302	TSecr=141019143				
7536	1207	9251580	10.0.0.1	10.0.0.1	TCP	68	43270 → 5000	[ACK] Seq=123	ACK=409	Win=2496	Len=0	TsVal=25101302	TSecr=141019143				
7536	1207	9252947	10.0.0.1	10.0.0.1	TCP	68	43270 → 5000	[ACK] Seq=123	ACK=409	Win=2496	Len=0	TsVal=25101302	TSecr=141019143				
7540	1207	9256551	10.0.0.1	10.0.0.5	TCP	79	5000 → 43592	[PSH, ACK] Seq=555354	ACK=122	Win=4350	Len=54	TsVal=141019143	TSecr=25101302				
7541	1207	9256551	10.0.0.1	10.0.0.5	TCP	79	TCP Retransmission	5000 → 43592	[PSH, ACK] Seq=555354	ACK=122	Win=4350	Len=54	TsVal=141019143	TSecr=25101302			
7541	1207	9256551	10.0.0.1	10.0.0.5	TCP	79	TCP Retransmission	5000 → 43592	[PSH, ACK] Seq=555354	ACK=122	Win=4350	Len=54	TsVal=141019143	TSecr=25101302			
7542	1207	9256796	10.0.0.5	10.0.0.2	TCP	68	43270 → 5000	[ACK] Seq=180	ACK=528	Win=2496	Len=0	TsVal=25128472	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.2	TCP	68	TCP Out of order	43270 → 5000	[ACK] Seq=180	ACK=528	Win=2496	Len=0	TsVal=25128472	TSecr=132609468			
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	68	TCP Out of order	43270 → 5000	[ACK] Seq=180	ACK=528	Win=2496	Len=0	TsVal=25128472	TSecr=132609468			
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	TSecr=132609468				
7542	1207	9256796	10.0.0.5	10.0.0.1	TCP	71	43592 → 5000	[PSH, ACK] Seq=180	ACK=528	Win=2496	Len=3	TsVal=92128474	T				

Fig. 7.3 — Post-authentication chat segments (PSH, ACK) flowing between the server and multiple connected clients

7.4 Logout / Session Teardown

[illegible]

Fig. 7.4 — The /logout exchange followed by the FIN, ACK teardown of the TCP connection

7.5 Observation on Retransmissions

The captures show a noticeable number of frames marked by Wireshark as “TCP Retransmission”, “TCP Dup ACK” and “TCP Out-of-Order” even though pingall reported 0% loss and the application behaved correctly end-to-end. This is consistent with a known Mininet/Open vSwitch artefact rather than genuine loss: virtual interfaces have TCP checksum offload enabled, so Wireshark computes an invalid checksum on outgoing segments it captures before the NIC/OVS layer fills it in, and its heuristic misclassifies these as retransmissions or out-of-order segments. The consistently rising TCP sequence numbers and successful application-level exchanges (correct AUTH responses, chat delivery, clean logout) confirm the underlying transfer was not actually impaired.

8. Conclusion

This assignment successfully layered practical application-level security onto the Assignment 6 TCP chat system without altering its core networking design. SHA-256 password hashing, duplicate-login prevention, structured input validation, a 5-attempt lockout policy, inactivity timeouts and an append-only security log were all implemented on the server and verified both functionally (via `security_log.txt` and `chat_history.csv`) and at the packet level (via Wireshark on `tcp.port == 5000`). The exercise reinforced how authentication, session state and logging can be built as a layer around an existing socket application, and highlighted a practical debugging skill — distinguishing genuine packet loss from Mininet/OVS checksum-offload artefacts in Wireshark captures.